

universidad de León Online

Universidad de León ▪ Máster Universitario en Ciberseguridad

Ingeniería del software seguro en la era de la IA generativa

Diseño y Programación Seguras

Evaluación reproducible de sistemas de IA

Francisco Javier Rodríguez Lera

Material elaborado con asistencia de IA generativa (DeepSeek-V4-Flash; considerados también GPT-6 y Claude Sonnet 4.6). Contenido revisado y validado por el autor.



Instituto de Investigación e Innovación en Ingeniería

Escuela de Ingenierías Industrial, Informática y Aeroespacial

universidad de León

Objetivos de aprendizaje

Al terminar el capítulo, el estudiante será capaz de:

- **Diseñar** un protocolo de evaluación reproducible (F0-F7).
- **Distinguir** los tres regímenes de evaluación y sus observables.
- **Identificar** variables controladas y variables confusas.
- **Definir** métricas como proxies y declarar sus amenazas a la validez.
- **Rechazar** la falacia «gratuito contra de pago» como criterio de seguridad.

Comparar es diseñar un experimento

Comparar sistemas de IA es un problema de *diseño experimental*: hay que declarar qué se mide, sobre qué unidad, con qué variables fijadas y con qué límites de inferencia. Sin ese control, la comparación degenera en anécdota, y una anécdota no es evidencia.

- Este capítulo responde a una pregunta previa a «¿qué propiedad tiene este código?»: «¿cómo se compara, de forma reproducible, lo que producen dos sistemas?».
- La IA no mejora ni empeora *per se*: desplaza el esfuerzo hacia la especificación, la revisión y la validación.
- La evaluación mide el **riesgo residual** de ese desplazamiento.

La comparación no es una propiedad del modelo

«El sistema A es más seguro que el sistema B» afirma algo sobre una *tupla* de condiciones, no sobre un modelo aislado.

- Modelo y **versión**.
- **Configuración** (*system prompt*, temperatura, filtros, permisos).
- *Prompt* y **contexto**.
- **Herramientas** habilitadas.
- **Entorno** de ejecución; repeticiones como covariable.

Consecuencia: el objeto de la evaluación no es el modelo, sino la *configuración concreta* en la que se usa.

La falacia de «gratuito contra de pago»

- **Evidencia parcial:** no existe relación funcional conocida entre la tarifa y la seguridad del código producido; la tarifa es un atributo comercial.
- La comparación «gratuito vs. de pago» está *confundida*: los grupos difieren en más de una variable a la vez (versión, *system prompt*, contexto, herramientas, filtros, cuotas, región).
- La seguridad de un sistema desplegado depende de la **configuración de despliegue**, no solo del modelo base.

El precio es, en el mejor de los casos, un indicador indirecto y ruidoso de capacidad; **no es un control de seguridad**.

Cómo tratar el coste en el protocolo

- El coste **no se elimina**: se *registra como covariable*.
- Se anota la tarifa vigente, la fecha de consulta, la moneda y las condiciones de uso.
- No se usa como la variable que explica el resultado.
- Si se quiere estudiar el efecto del precio, hay que manipularlo dejando constantes las demás variables —lo que con servicios cerrados casi nunca es posible—.

Reconocer esa imposibilidad *es parte del rigor*.

Unidad de análisis: tres regímenes

Régimen A - Generación aislada. Recibe un *prompt* y devuelve un artefacto. No ejecuta, no escribe, no accede a red. Observable: el artefacto. Reproducibilidad alta.

Régimen B - Agente con herramientas. Bucle de decisión con herramientas (comandos, ficheros, red, paquetes). Observable: artefacto *y acciones*. Superficie de ataque crece (inyección de *prompt*, cadena de suministro, secretos).

Régimen C - Pipeline de validación. El artefacto o las acciones se someten a análisis estático, pruebas dinámicas y *sandbox*. Decide si el resultado es aceptable.

Regímenes de evaluación

Dimensión	A. Generación aislada	B. Agente con herramientas	C. Pipeline
Observable	Artefacto (texto/código)	Artefacto y acciones	SAST, pruebas, *sandbox*
Efectos secundarios	Ninguno	Escrituras, red, paquetes	Según aislamiento
Reproducibilidad	Alta (salvo muestreo)	Media o baja	Depende de herramientas fijadas
Amenaza principal	Vulnerabilidad en el artefacto	Inyección, cadena de suministro, secretos	Falsos positivos/negativos
Control asociado	Revisión de código	*Sandbox* y límites de confianza	Puertas de calidad

¿Se pueden comparar regímenes distintos?

- Un número del régimen A («cuántos requisitos cumple el fragmento»), ¿es comparable con uno del régimen B («cuántas acciones peligrosas ejecutó el agente»)?

Variables controladas del protocolo

Variable	Cómo se fija	Riesgo si no se controla
Versión	ID exacto del modelo, fecha, proveedor y punto de acceso	Deriva silenciosa
Configuración	*System prompt*, temperatura, filtros, permisos	Diferencia atribuible a config.
Temperatura	Se fija y declara; cero *no* garantiza determinismo	Variabilidad no declarada
Prompt	Texto literal; ID y contenido íntegro	Reformulación cambia resultado
Contexto	Ficheros/fragmentos; política de recuperación	Forma parte del tratamiento
Herramientas	Lista, versiones, permisos, red	Régimen B no declarado
Entorno	SO, arquitectura, imagen, *sandbox*	Fallo de plataforma
Repeticiones	n por celda, orden, semilla	Ruido confundido con efecto

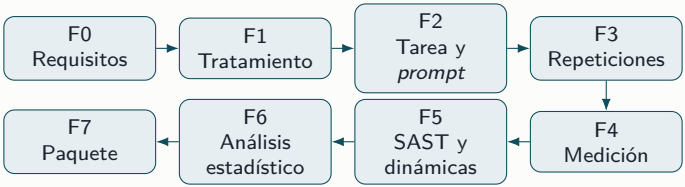
Variables confusas típicas

Toda variable no registrada es una variable confusa en potencia.

- Tarifa y condiciones del servicio.
- Proveedor y punto de acceso (interfaz web frente a API).
- Fecha y hora (por la deriva del modelo).
- Región de despliegue y cuotas de uso.
- Idioma del *prompt*.
- Experiencia del revisor humano que evalúa.

Ninguna es el tratamiento, pero cualquiera puede correlacionar con él.

Diseño del protocolo: fases F0-F7



Cada fase produce un registro que alimenta el paquete de reproducibilidad (F7).

Las fases del protocolo, en detalle

- **F0 - Requisitos:** especificación congelada, con identificadores estables (caso transversal + ASVS [3] y SSDF [2] como guía; CWE [1] para clasificar).
- **F1 - Tratamiento:** sistemas a comparar + valor de cada variable; incluye *brazo de control* (solución humana de referencia y solución vulnerable).
- **F2 - Tarea y *prompt*:** un *prompt* único, literal e idéntico.
- **F3 - Repeticiones:** n por celda; orden aleatorizado o contrarrestado; semilla.
- **F4 - Medición:** métricas sobre artefactos y acciones; ciega al origen si es posible.
- **F5 - SAST y dinámicas:** convierte A/B en régimen C.
- **F6 - Análisis estadístico** y **F7 - Paquete de reproducibilidad**.

Métricas: definiciones operativas (proxies)

Métrica	Definición operativa	Escala
Requisitos satisfechos	Cumplidos sobre el total de F0	Recuento/fracción
Vulnerabilidades	Fallos reproducidos por CWE y severidad	Recuento por severidad
Portabilidad	Forma medida y plataformas donde se cumple	Recuento por forma
Dependencias	Número, origen y versión	Recuento y origen
Complejidad	LOC, ciclomática, anidamiento	Medida cuantitativa
SAST	Hallazgos por regla, normalizados	Recuento por regla
Pruebas dinámicas	Cobertura, *fuzzing*, saneadores	Fracción/estado
Coste	Consumo y tarifa (con fecha y moneda)	Monetario/consumo
Tiempo de revisión	Tiempo humano con protocolo de parada	Duración

Las métricas son proxies

- Una métrica es una **definición operativa**: mide un *proxy* observable, no «seguridad» directamente.
- La severidad es una *convención*: solo comparable si se fija antes de medir y se aplica igual a todos; si dos revisores la aplican, se mide su acuerdo.
- No existe una severidad «objetiva» independiente del procedimiento que la define.

Superar una batería finita de pruebas no demuestra seguridad. «Pruebas superadas» es un resultado del instrumento, no una propiedad del artefacto.

Tratamiento estadístico (Propuesta docente)

- **Unidad e independencia:** medidas repetidas y anidadas; tratar repeticiones como independientes infla la muestra efectiva.
- **Descriptiva primero:** distribución completa, mediana y rango intercuartílico con datos sesgados; siempre reportar n, dispersión y extremos.
- **Tamaño de muestra:** justificar n antes de ejecutar (potencia o piloto).
- **Diseño emparejado** cuando varios sistemas resuelven la misma tarea.
- **Tamaño del efecto e intervalos de confianza**, no solo valor p.
- **Comparaciones múltiples:** métrica principal preespecificada; corrección o etiqueta de exploratorio.
- **No determinación:** la variabilidad residual se mide, no se supone.
- **Fallos y faltantes** son datos; **pre-registro** para evitar HARKing.

Amenazas a la validez

Tipo de validez	Amenazas que el protocolo declara
Interna	Confusión, deriva del modelo, no determinación, efectos de orden y aprendizaje, contaminación
De constructo	«Requisitos satisfechos» es proxy; SAST incluye falsos positivos y omite falsos negativos; severidad es convención
Externa	Un caso, un lenguaje, una tarea, un dominio; la validez ecológica exige preguntar si el flujo se parece al uso real
De conclusión	Muestras pequeñas, comparaciones múltiples, observaciones no independientes, reporte selectivo
Reproducibilidad	Actualizaciones silenciosas, deriva de herramientas, sesgo del evaluador; un resultado no se transfiere sin reevaluar

¿Basta con un *pipeline* verde?

- Si un artefacto pasa todas las comprobaciones del protocolo, ¿queda demostrada su seguridad?

Síntesis del capítulo

- **Lo que se compara no es el modelo, sino una configuración concreta**, y solo es comparable si se fijan y registran todas sus variables.
- **El precio no es una variable de seguridad**: tratarlo como causa es un error de confusión.
- **Hay que distinguir los regímenes**: confundirlos es un error de constructo.
- **Las métricas son proxies**: acotan el riesgo observado; no demuestran seguridad.

La sección ha definido un protocolo, no un resultado.

Actividad: pre-registro de dos páginas

1. Enumerad los requisitos funcionales y de seguridad del caso transversal con identificadores estables.
2. Rellenad una ficha de sesión para dos sistemas que queráis comparar.
3. Definid la métrica principal y el número de repeticiones.
4. Enumerad al menos cinco amenazas a la validez y cómo mitigarlas o declararlas.

¿Qué diferencia una anécdota de una evidencia?

- «A mí me fue mejor con el sistema B», ¿es evidencia de que B es más seguro?

Referencias I



MITRE Corporation.

Common weakness enumeration (cwe).

Catálogo de referencia; los identificadores deben justificarse por caso.



National Institute of Standards and Technology.

Secure software development framework (ssdf) version 1.1: Recommendations for mitigating the risk of software vulnerabilities.

Technical Report SP 800-218, NIST, 2022.

Norma / marco de referencia.



OWASP Foundation.

Application security verification standard (asvs), 2025.

Guía de referencia (versión estable 5.0.0, 2025).